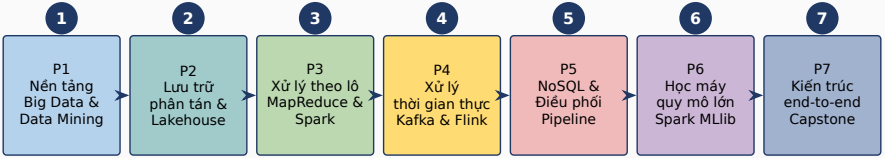
BIG DATA ANALYTICS

Tuần 1 — Roadmap & Thiết lập môi trường

Giới thiệu tổng quan 7 phần nội dung học phần
và hướng dẫn cài đặt công cụ thực hành



Biên soạn tham khảo trên nền đề cương BDAN437764 — Big Data Analytics

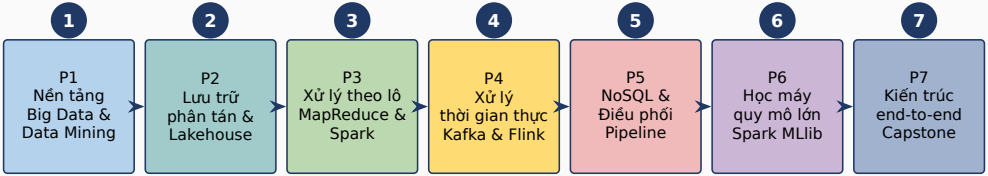
Khoa Đào tạo Chất lượng cao — Trường Đại học Sư phạm Kỹ thuật TP.HCM

Nội dung buổi học Tuần 1

- Ngữ cảnh & ứng dụng: những nơi Big Data thực sự tạo ra giá trị
- Roadmap tổng quan: 7 phần nội dung sẽ học trong 9 tuần
- Giới thiệu nhanh từng phần trong roadmap (7 phần)
  - Vấn đề mỗi phần giải quyết
  - Công nghệ cốt lõi
  - Mối liên kết với các phần khác
- Thực hành: thiết lập môi trường công cụ cho toàn khóa học
- Tổng kết & chuẩn bị cho Tuần 2

NGỮ CẢNH & ỨNG DỤNG CỦA BIG DATA

Trước khi học công nghệ, hãy hiểu Big Data tạo ra giá trị ở đâu



Vì sao bắt đầu từ ứng dụng thay vì công nghệ?

- Công nghệ (Hadoop, Spark, Kafka...) chỉ là công cụ — câu hỏi quan trọng hơn là **bài toán nào cần đến chúng**
- Hiểu ngữ cảnh giúp trả lời: vì sao không dùng Excel hay một máy chủ SQL bình thường?
- Mỗi ứng dụng dưới đây sẽ được nhắc lại trong các phần kỹ thuật (P1–P7) như một sợi chỉ xuyên suốt khóa học

Ứng dụng 1: Thương mại điện tử

- Hệ thống gợi ý sản phẩm (recommendation engine) phân tích hành vi click, xem, mua của hàng triệu người dùng mỗi ngày
- Đặc trưng nổi bật: **Volume** (log hành vi khổng lồ) và **Velocity** (cập nhật gợi ý gần thời gian thực)
- Sẽ quay lại ở Phần 1 (kỹ thuật khai phá dữ liệu) và Phần 6 (học máy quy mô lớn)



Ứng dụng 2: Mạng xã hội

- Xếp hạng newsfeed, phát hiện tin giả/nội dung độc hại trên hàng tỷ bài đăng
- Dữ liệu dạng đồ thị quan hệ (ai kết nối với ai) — đặc trưng **Variety** rất cao
- Sẽ quay lại ở Phần 5 khi bàn về cơ sở dữ liệu đồ thị (graph database)



Ứng dụng 3: Tài chính — Ngân hàng

- Phát hiện gian lận giao dịch thẻ **ngay khi giao dịch xảy ra**, không thể chờ xử lý theo lô
- Đặc trưng nổi bật: **Velocity** (độ trễ phải dưới vài trăm mili-giây) và **Veracity** (độ tin cậy quyết định)
- Sẽ quay lại ở Phần 4 (xử lý dữ liệu thời gian thực)



Ứng dụng 4: Y tế

- Phân tích hồ sơ bệnh án điện tử, ảnh y khoa, dữ liệu giải trình tự gen phục vụ nghiên cứu dịch tễ học
- Đặc trưng nổi bật: **Variety** (văn bản, hình ảnh, tín hiệu) và **Veracity** (sai số có thể ảnh hưởng sinh mạng)
- Đòi hỏi lưu trữ đa dạng và bảo mật cao — liên hệ Phần 2 và Phần 5



Ứng dụng 5: Giao thông & Logistics

- Tối ưu tuyến đường, dự đoán nhu cầu di chuyển/vận chuyển theo thời gian thực (ví dụ các nền tảng gọi xe, giao hàng)
- Kết hợp cả dữ liệu streaming (vị trí GPS) và dữ liệu lịch sử (batch) để dự đoán
- Ví dụ điển hình cho kiến trúc Lambda/Kappa sẽ học ở Phần 7



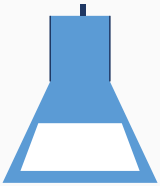
Ứng dụng 6: IoT & Thành phố thông minh

- Hàng triệu cảm biến (giao thông, môi trường, năng lượng) liên tục phát sinh dữ liệu thời gian thực
- Đặc trưng nổi bật: **Velocity** cực cao, dữ liệu dạng chuỗi thời gian (time-series)
- Liên hệ trực tiếp Phần 4 (Kafka/Flink xử lý luồng cảm biến)



Ứng dụng 7: Khoa học

- Thiên văn học (dữ liệu kính viễn vọng), vật lý hạt (CERN), giải trình tự gen — quy mô dữ liệu tới hàng petabyte
- Đặc trưng nổi bật: **Volume** cực lớn, cần lưu trữ và tính toán phân tán hiệu quả
- Là động lực lịch sử ra đời của GFS/Hadoop — sẽ nhắc lại ở Phần 2



Ứng dụng 8: Nền tảng Streaming Media

- Cá nhân hóa nội dung đề xuất (phim, nhạc, video) dựa trên lịch sử xem của từng người dùng
- Kết hợp học máy quy mô lớn (Phần 6) với xử lý sự kiện thời gian thực (Phần 4)
- Minh họa rõ nét việc một sản phẩm thực tế cần **toàn bộ** 7 phần trong roadmap



Điểm chung: đặc trưng 5V nào nổi bật nhất?

Ứng dụng	Đặc trưng 5V nổi bật	Phần liên quan
Thương mại điện tử	Volume, Velocity	P1, P6
Mạng xã hội	Variety, Volume	P1, P5
Tài chính — gian lận	Velocity, Veracity	P4
Y tế	Variety, Veracity	P2, P5
Giao thông/Logistics	Velocity, Volume	P4, P7
IoT / Thành phố thông minh	Velocity	P4
Khoa học	Volume	P2, P3
Streaming media	Volume, Velocity, Value	P4, P6

Vì sao công cụ truyền thống không đủ?

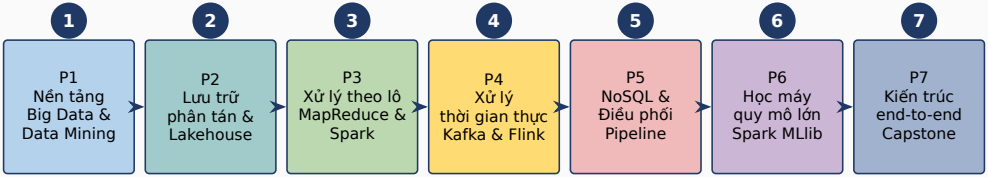
- Một máy chủ SQL đơn lẻ giới hạn bởi RAM/CPU/đĩa của **một máy** — không thể mở rộng vô hạn (scale-up có trần)
- Excel/công cụ văn phòng không xử lý được dữ liệu phi cấu trúc (hình ảnh, văn bản tự do, luồng sự kiện)
- Các bài toán trên đòi hỏi: mở rộng theo chiều ngang (scale-out trên nhiều máy), xử lý gần thời gian thực, và lưu trữ đa định dạng
- $\Rightarrow$ Đây chính là lý do ra đời của toàn bộ hệ sinh thái công nghệ Big Data mà 7 phần sắp tới sẽ trình bày

Từ ứng dụng đến công nghệ

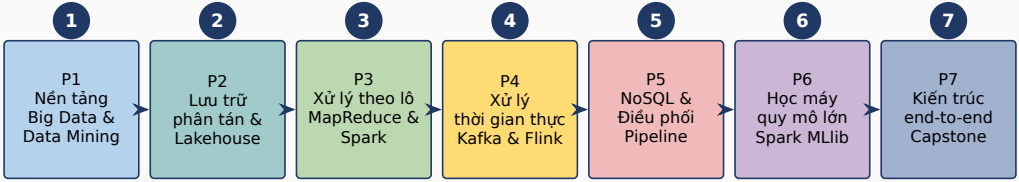
- Mỗi ứng dụng ở trên không dùng một công nghệ đơn lẻ, mà là **sự kết hợp** nhiều phần trong roadmap
- Roadmap 7 phần sắp giới thiệu chính là bộ công cụ để xây dựng nên các hệ thống như trên
- Hãy giữ những ví dụ này trong đầu — chúng ta sẽ quay lại chúng nhiều lần trong suốt khóa học

ROADMAP 7 PHẦN NỘI DUNG HỌC PHẦN

Bức tranh toàn cảnh của cả khóa học, trước khi đi vào chi tiết



Toàn cảnh Roadmap



7 phần được thiết kế theo trình tự logic: mỗi phần là nền tảng cho phần tiếp theo

Logic kết nối giữa 7 phần

- P1 → P2 → P3: trục xương sống
  - Hiểu khái niệm (P1) → biết lưu ở đâu (P2) → biết xử lý thế nào (P3)
- P3 → P4, P3 → P5: mở rộng theo hai hướng
  - P4 mở rộng sang xử lý thời gian thực
  - P5 mở rộng sang lưu trữ đa mô hình và điều phối pipeline
- P1–P5 → P6: nâng cấp lên phân tích dự đoán
  - Toàn bộ hạ tầng trước đó phục vụ cho việc huấn luyện mô hình học máy
- Tất cả → P7: hội tụ thành kiến trúc hoàn chỉnh
  - Phần 7 không giới thiệu công nghệ mới, mà dạy cách **phối hợp** mọi công nghệ đã học

Cấu trúc 9 tuần học

Tuần	Nội dung
Tuần 1	Roadmap tổng quan + Thiết lập môi trường (buổi học hôm nay)
Tuần 2	Phần 1 — Nền tảng Big Data & Data Mining
Tuần 3	Phần 2 — Lưu trữ phân tán hiện đại
Tuần 4	Phần 3 — Xử lý theo lô (MapReduce & Spark)
Tuần 5	Phần 4 — Xử lý dữ liệu thời gian thực
Tuần 6	Phần 5 — NoSQL & Điều phối Pipeline
Tuần 7	Phần 6 — Học máy quy mô lớn
Tuần 8	Phần 7 — Kiến trúc ứng dụng end-to-end (Capstone)
Tuần 9	Ôn tập & Thi

Cách học hiệu quả theo roadmap

- Mỗi tuần: 3 tiết lý thuyết (hiểu khái niệm & kiến trúc) + 2 tiết thực hành (lập trình/thao tác công cụ)
- Trước mỗi tuần: đọc trước tài liệu tham khảo được liệt kê ở slide cuối mỗi phần
- Sau mỗi phần: tự trả lời các câu hỏi ôn tập ở slide tổng kết để củng cố kiến thức
- Dự án capstone (Phần 7 / Tuần 8) nên được nhóm bắt đầu hình thành ý tưởng từ sớm — càng sớm càng tốt

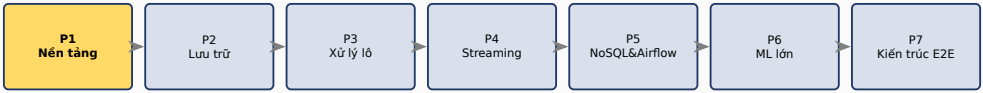
Sẵn sàng bắt đầu

- Chúng ta đã thấy **vì sao** cần Big Data (ngữ cảnh & ứng dụng)
- Chúng ta đã thấy **cái gì** sẽ được học (roadmap 7 phần)
- Tiếp theo: đi sâu vào từng phần, bắt đầu từ nền tảng — **Phần 1**

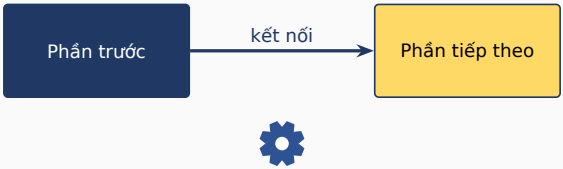
PHẦN 1

NỀN TẢNG BIG DATA & DATA MINING

Khái niệm cốt lõi và kỹ thuật khai phá dữ liệu làm nền cho toàn bộ khóa học



Kết nối từ phần Ngữ cảnh & Ứng dụng



Chúng ta vừa thấy Big Data hiện diện ở khắp mọi lĩnh vực. Phần 1 sẽ trả lời hai câu hỏi nền tảng:

- Big Data thực chất là gì — khác gì với "nhiều dữ liệu" thông thường?
- Data mining — kỹ thuật rút ra tri thức từ dữ liệu — đóng vai trò gì trong bức tranh đó?

Mục tiêu học tập của Phần 1

- Trình bày và phân loại được khái niệm, đặc trưng của Big Data
- Phân biệt Big Data với hệ thống dữ liệu truyền thống
- Nắm được 4 kỹ thuật khai phá dữ liệu nền tảng: classification, clustering, association rules, regression
- Nhận diện được kỹ thuật phù hợp cho một bài toán thực tế cụ thể

Định nghĩa Big Data

- Big Data: tập dữ liệu có quy mô, tốc độ phát sinh hoặc mức độ đa dạng **vượt quá khả năng xử lý** của các công cụ quản trị dữ liệu truyền thống (một máy chủ, một CSDL quan hệ)
- Khái niệm "3V" gốc (Volume, Velocity, Variety) được Doug Laney (Gartner) đề xuất năm 2001
- Sau này mở rộng thành 5V khi ngành công nghiệp nhận ra chất lượng và giá trị dữ liệu cũng quan trọng không kém quy mô

Năm đặc trưng của Big Data (5V)



5V không phải là định nghĩa cứng nhắc, mà là khung tư duy để đánh giá một bài toán có "đủ Big" hay không

Volume & Velocity

Volume (khối lượng)

- Quy mô dữ liệu vượt khả năng lưu trữ/xử lý của một máy đơn lẻ
- Đo bằng terabyte, petabyte, thậm chí exabyte
- Ví dụ: log hành vi người dùng của một sàn TMĐT lớn tích lũy hàng ngày

Velocity (tốc độ)

- Tốc độ dữ liệu phát sinh và cần được xử lý
- Có thể là tốc độ nạp (ingestion) hoặc tốc độ cần phản hồi (latency)
- Ví dụ: giao dịch thẻ ngân hàng cần được đánh giá gian lận trong vài trăm mili-giây

Variety & Veracity

Variety (đa dạng)

- Dữ liệu có cấu trúc (bảng), bán cấu trúc (JSON, log), phi cấu trúc (ảnh, văn bản, video)
- Đến từ nhiều nguồn khác nhau: cảm biến, mạng xã hội, giao dịch, hệ thống nội bộ

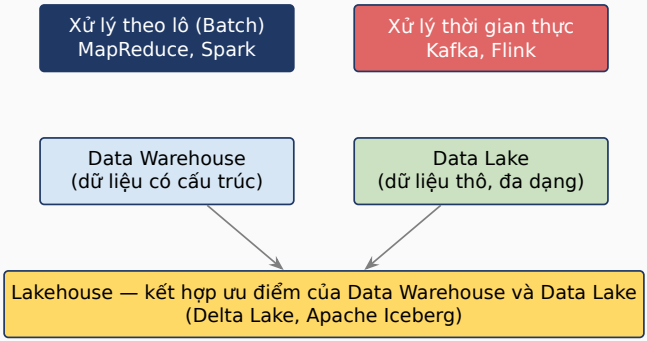
Veracity (độ tin cậy)

- Chất lượng và độ chính xác của dữ liệu — dữ liệu nhiễu, thiếu, hoặc sai lệch
- Ví dụ: dữ liệu cảm biến IoT có thể bị lỗi phần cứng, dữ liệu mạng xã hội có thể là tin giả

Value — chữ V quan trọng nhất

- Bốn chữ V đầu mô tả **đặc điểm** của dữ liệu; Value mô tả **mục đích** thu thập và xử lý nó
- Dữ liệu chỉ có giá trị khi được chuyển hóa thành tri thức hoặc quyết định hữu ích — đây chính là vai trò của **Big Data Analytics**, mục tiêu xuyên suốt học phần này
- Toàn bộ 7 phần trong roadmap là các bước kỹ thuật để đi từ dữ liệu thô đến giá trị (Value)

Toàn cảnh hệ sinh thái công nghệ Big Data



Hai trục phân loại chính: cách xử lý (batch/streaming) và cách lưu trữ (warehouse/lake/lakehouse) — sẽ được trình bày chi tiết ở Phần 2–4

Vì sao hệ thống truyền thống (RDBMS) không đủ?

- Cơ sở dữ liệu quan hệ (MySQL, PostgreSQL...) được thiết kế để **scale-up**: nâng cấp phần cứng của một máy duy nhất — có giới hạn vật lý
- Big Data cần **scale-out**: phân tán dữ liệu và tính toán trên nhiều máy rẻ tiền, mở rộng gần như không giới hạn
- Đánh đổi: hệ thống phân tán phức tạp hơn về mặt vận hành, nhất quán dữ liệu, và chịu lỗi — đây là lý do cả một hệ sinh thái công nghệ (Phần 2–7) ra đời

Data Mining là gì?

- Data mining: quá trình khám phá các mẫu (pattern), quy luật hoặc tri thức ẩn trong tập dữ liệu lớn
- Là một bước trong quy trình rộng hơn gọi là **KDD** (Knowledge Discovery in Databases): thu thập $\rightarrow$ tiền xử lý $\rightarrow$ khai phá $\rightarrow$ đánh giá $\rightarrow$ trực quan hóa
- Trong bối cảnh Big Data, data mining phải chạy được trên hạ tầng phân tán (đây là cầu nối sang Phần 3 và Phần 6)

Kỹ thuật 1 — Classification (Phân loại)

- Gán một nhãn/lớp đã biết trước cho dữ liệu mới, dựa trên dữ liệu đã gán nhãn (học có giám sát)
- Ví dụ: phân loại email là spam hay không spam, dựa trên các email đã được gán nhãn trước đó
- Thuật toán tiêu biểu: Decision Tree, Naive Bayes, Logistic Regression — sẽ gặp lại ở Phần 6 (Spark MLlib)

Kỹ thuật 2 — Clustering (Phân cụm)

- Nhóm các đối tượng có đặc điểm tương tự vào cùng một cụm, **không cần** nhãn trước (học không giám sát)
- Ví dụ: phân khúc khách hàng theo hành vi mua sắm để cá nhân hóa chiến dịch marketing
- Thuật toán tiêu biểu: K-Means — sẽ gặp lại ở Phần 6

Kỹ thuật 3 — Association Rules (Luật kết hợp)

- Tìm các mối quan hệ "nếu... thì..." xuất hiện thường xuyên trong dữ liệu giao dịch
- Ví dụ kinh điển: phân tích giỏ hàng (market-basket analysis) — khách mua bỉm thường mua kèm bia (trường hợp nổi tiếng trong tài liệu ngành bán lẻ)
- Ứng dụng: gợi ý sản phẩm đi kèm, bố trí kệ hàng, khuyến mãi combo

Kỹ thuật 4 — Regression (Hồi quy)

- Dự đoán một giá trị số liên tục dựa trên các biến đầu vào
- Ví dụ: dự đoán doanh thu tháng tới dựa trên dữ liệu bán hàng lịch sử và các yếu tố mùa vụ
- Là nền tảng toán học cho nhiều mô hình học máy phức tạp hơn ở Phần 6

So sánh 4 kỹ thuật khai phá dữ liệu

Kỹ thuật	Loại học	Đầu ra	Ví dụ
Classification	Có giám sát	Nhãn rời rạc	Lọc email spam
Clustering	Không giám sát	Nhóm/cụm	Phân khúc khách hàng
Association Rules	Không giám sát	Luật kết hợp	Gợi ý sản phẩm đi kèm
Regression	Có giám sát	Giá trị số liên tục	Dự đoán doanh thu

Case study: áp dụng cả 4 kỹ thuật cho một sàn TMĐT



- **Classification**: dự đoán khách hàng có rời bỏ (churn) hay không
- **Clustering**: phân khúc khách hàng theo hành vi mua sắm
- **Association rules**: gợi ý "sản phẩm thường mua kèm"
- **Regression**: dự đoán doanh thu theo mùa vụ
- Một hệ thống thực tế thường kết hợp **nhiều** kỹ thuật cùng lúc, không chỉ một

Thách thức khi khai phá dữ liệu ở quy mô lớn

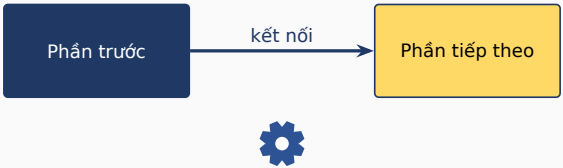
- Thuật toán data mining truyền thống thường giả định dữ liệu nằm gọn trong bộ nhớ một máy — không còn đúng ở quy mô Big Data
- Cần các phiên bản thuật toán **phân tán** chạy song song trên nhiều máy (chủ đề của Phần 3 và Phần 6)
- Chất lượng dữ liệu (Veracity) ảnh hưởng trực tiếp đến độ tin cậy của kết quả khai phá — "garbage in, garbage out"

Vị trí của Phần 1 trong Roadmap

- Phần 1 là **nền tảng khái niệm** cho toàn bộ 6 phần còn lại
- Mọi công nghệ sắp học (HDFS, Spark, Kafka...) đều là công cụ để giải quyết các vấn đề Volume/Velocity/Variety vừa nêu
- Các kỹ thuật data mining vừa học sẽ được "phóng to" lên quy mô lớn ở Phần 6



Cầu nối sang Phần 2



*Đã hiểu **khái niệm** và **kỹ thuật** — nhưng trước khi tính toán trên dữ liệu lớn, câu hỏi thực tế đầu tiên là:*

- Dữ liệu hàng petabyte đó sẽ được **lưu trữ ở đâu**?
- Làm sao đảm bảo dữ liệu không mất khi một máy trong cluster gặp sự cố?
- $\rightarrow$ Phần 2 trả lời bằng HDFS, Cloud Object Storage và kiến trúc Lakehouse

Tổng kết Phần 1

Những điểm mấu chốt:

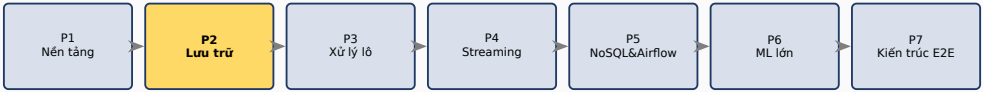
- Big Data được đặc trưng bởi 5V: Volume, Velocity, Variety, Veracity, Value
- RDBMS truyền thống giới hạn ở scale-up; Big Data cần scale-out
- 4 kỹ thuật khai phá dữ liệu nền tảng: classification, clustering, association rules, regression
- Data mining là một bước trong quy trình KDD rộng hơn

Câu hỏi ôn tập

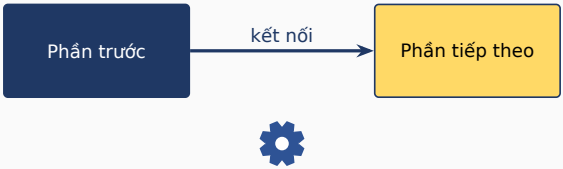
1. Phân biệt Volume và Velocity bằng ví dụ của riêng bạn
2. Một bài toán dự đoán điểm thi của sinh viên nên dùng kỹ thuật nào trong 4 kỹ thuật đã học? Vì sao?
3. Vì sao "nhiều dữ liệu" chưa chắc đã là "Big Data"?

PHẦN 2
LƯU TRỮ PHÂN TÁN HIỆN ĐẠI

HDFS, Cloud Object Storage, định dạng cột và kiến trúc Lakehouse



Kết nối từ Phần 1



Phần 1 đã xác định: dữ liệu lớn cần scale-out trên nhiều máy. Phần 2 trả lời câu hỏi thực tế đầu tiên:

- Dữ liệu hàng petabyte sẽ được lưu trữ vật lý như thế nào?
- Làm sao đảm bảo không mất dữ liệu khi phần cứng hỏng?

Mục tiêu học tập của Phần 2

- Hiểu nguyên lý hệ thống tệp phân tán (HDFS) và vai trò của YARN
- Hiểu xu hướng dịch chuyển sang lưu trữ đối tượng trên cloud
- Hiểu vì sao định dạng lưu trữ dạng cột tối ưu hiệu năng truy vấn
- Hiểu kiến trúc Lakehouse và lý do nó đang thay thế Data Lake truyền thống

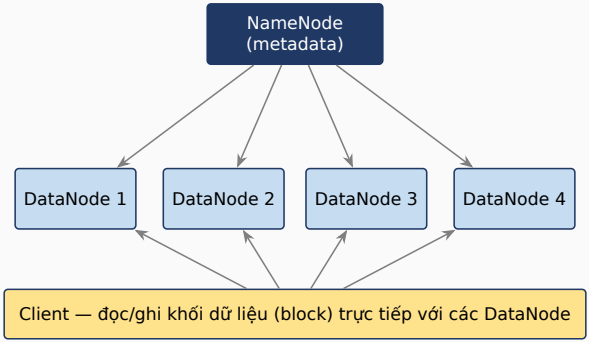
Vấn đề: một máy không đủ chứa dữ liệu lớn

- Ổ đĩa của một máy chủ có giới hạn (vài chục TB) — không đủ cho dữ liệu hàng petabyte
- Ngay cả khi đủ dung lượng, tốc độ đọc/ghi của một đĩa cứng là điểm nghẽn (bottleneck) khi nhiều tiến trình cùng truy cập
- Giải pháp: chia nhỏ dữ liệu và trải ra trên **nhiều máy** — đây là ý tưởng cốt lõi của hệ thống tệp phân tán

Google File System (GFS) — ý tưởng gốc

- Công bố năm 2003 bởi Google (Ghemawat, Gobioff, Leung) để giải quyết chính vấn đề lưu trữ dữ liệu web quy mô lớn
- Ý tưởng cốt lõi: chia file thành các **khối (chunk)** lớn, nhân bản (replicate) mỗi khối trên nhiều máy để chịu lỗi
- Apache Hadoop Distributed File System (HDFS) là hiện thực mã nguồn mở lấy cảm hứng trực tiếp từ GFS

Kiến trúc HDFS



NameNode quản lý metadata (vị trí các khối); DataNode lưu trữ dữ liệu thực tế và phục vụ Client trực tiếp

NameNode vs. DataNode

NameNode

- Lưu **metadata**: cấu trúc thư mục, tên file, vị trí các khối dữ liệu
- Là điểm điều phối trung tâm — không lưu dữ liệu thực tế
- Thường có NameNode dự phòng (High Availability) để tránh single point of failure

DataNode

- Lưu trữ **dữ liệu thực tế** dưới dạng các khối (block, mặc định 128MB)
- Định kỳ báo cáo tình trạng (heartbeat) về NameNode
- Có thể có hàng nghìn DataNode trong một cluster lớn

Cơ chế chia khối & nhân bản (Replication)

- Mỗi file lớn được chia thành nhiều **block** kích thước cố định (mặc định 128MB trong Hadoop)
- Mỗi block được nhân bản (mặc định 3 bản sao) trên các DataNode khác nhau, thường ở các rack vật lý khác nhau
- Nếu một DataNode hỏng, NameNode phát hiện qua heartbeat và tự động tạo lại bản sao ở nơi khác $\Rightarrow$ chịu lỗi (fault tolerance) mà không mất dữ liệu

YARN — Quản lý tài nguyên cluster

- YARN (Yet Another Resource Negotiator): tầng quản lý tài nguyên (CPU, RAM) tách biệt khỏi tầng lưu trữ (HDFS)
- Trước Hadoop 2.0, MapReduce vừa xử lý logic vừa quản lý tài nguyên — kém linh hoạt
- YARN cho phép nhiều framework xử lý khác nhau (MapReduce, Spark...) cùng chia sẻ tài nguyên trên một cluster

Giới hạn của HDFS truyền thống & xu hướng Cloud

- Cluster HDFS tự quản (on-premise) đòi hỏi đầu tư phần cứng cố định, khó co giãn (scale) linh hoạt theo nhu cầu thực tế
- Chi phí vận hành, bảo trì đội ngũ hạ tầng riêng là gánh nặng với nhiều tổ chức
- Xu hướng hiện nay: chuyển sang lưu trữ đối tượng (object storage) trên các nền tảng cloud, trả tiền theo nhu cầu sử dụng

Cloud Object Storage



Amazon S3 / Google Cloud Storage
/ Azure Blob Storage

Amazon S3, Google Cloud Storage, Azure Blob Storage — dữ liệu được lưu tập trung, nhiều cluster tính toán có thể cùng truy cập

Tách rời lưu trữ — tính toán (Storage-Compute Separation)

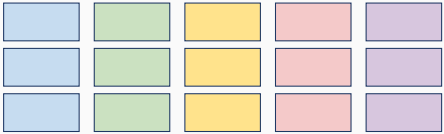
- Trong HDFS truyền thống, lưu trữ và tính toán nằm **trên cùng** các máy trong cluster
- Với object storage, dữ liệu nằm độc lập; các cluster tính toán (Spark, v.v.) có thể được tạo/xóa linh hoạt mà không ảnh hưởng dữ liệu
- Lợi ích: co giãn độc lập, tiết kiệm chi phí (tắt cluster tính toán khi không dùng), nhiều nhóm cùng chia sẻ một nguồn dữ liệu

Định dạng lưu trữ: Row-oriented vs. Column-oriented

Row-oriented (CSV)



Column-oriented (Parquet/ORC)



Lưu theo cột giúp nén tốt hơn (giá trị cùng loại đứng cạnh nhau) và chỉ đọc đúng cột cần truy vấn

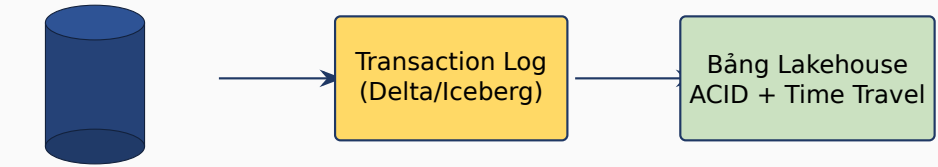
So sánh Parquet / ORC / Avro

Định dạng	Kiểu lưu	Thế mạnh
Parquet	Cột (columnar)	Truy vấn phân tích (OLAP), nén tốt, phổ biến nhất với Spark
ORC	Cột (columnar)	Tối ưu cho hệ sinh thái Hive, hỗ trợ index nội bộ
Avro	Hàng (row-based)	Phù hợp ghi tuần tự, truyền dữ liệu (ví dụ qua Kafka)

Vấn đề của Data Lake thuần túy

- Data Lake ban đầu: chỉ là nơi đổ file thô (Parquet/CSV/JSON) lên object storage, không có kiểm soát giao dịch
- Hệ quả: khó **cập nhật/xóa** dữ liệu an toàn, không đảm bảo tính nhất quán khi nhiều tiến trình ghi đồng thời — hiện tượng gọi là "data swamp"
- Cần một tầng quản lý giao dịch ở trên Data Lake $\Rightarrow$ đó là ý tưởng của **Lakehouse**

Kiến trúc Lakehouse



Transaction log (Delta Lake/Iceberg) thêm tính năng ACID lên trên các file dữ liệu thô của Data Lake

So sánh Delta Lake / Apache Iceberg / Apache Hudi

Công nghệ	Khởi xướng bởi	Đặc điểm nổi bật
Delta Lake	Databricks	Tích hợp sâu với Spark, transaction log dạng JSON
Apache Iceberg	Netflix	Trung lập engine (Spark/Flink/Trino đều dùng được), quản lý schema mạnh
Apache Hudi	Uber	Tối ưu cho upsert/incremental ingestion tần suất cao

Minh họa: Time Travel với Delta Lake

```
-- Truy vấn phiên bản hiện tại
SELECT * FROM sales;

-- Truy vấn dữ liệu tại một phiên bản cũ (time travel)
SELECT * FROM sales VERSION AS OF 12;
SELECT * FROM sales TIMESTAMP AS OF '2026-01-01';

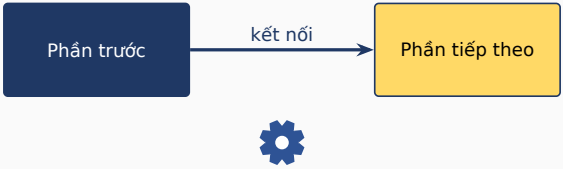
-- Thay đổi schema an toàn (schema evolution)
ALTER TABLE sales ADD COLUMN discount DOUBLE;
```

Vị trí của Phần 2 trong Roadmap

- Phần 2 xây dựng **nền lưu trữ** mà mọi phần xử lý phía sau (P3, P4, P6) sẽ đọc/ghi lên đó
- Lakehouse (Delta Lake/Iceberg) sẽ là điểm hội tụ dữ liệu batch và streaming ở Phần 7



Cầu nối sang Phần 3



Dữ liệu giờ đã có nơi lưu trữ đáng tin cậy và hiệu năng cao. Câu hỏi tiếp theo:

- Làm sao **tính toán** trên hàng petabyte dữ liệu đó trong thời gian hợp lý?
- $\rightarrow$ Phần 3 giới thiệu hai mô hình xử lý phân tán theo lô: MapReduce và Apache Spark

Tổng kết Phần 2

Những điểm mấu chốt:

- HDFS: NameNode quản lý metadata, DataNode lưu dữ liệu, chia khối & nhân bản để chịu lỗi
- Xu hướng cloud: tách rời lưu trữ và tính toán (object storage)
- Định dạng cột (Parquet/ORC) tối ưu hiệu năng truy vấn phân tích
- Lakehouse (Delta Lake/Iceberg/Hudi) thêm ACID, time travel, schema evolution lên Data Lake

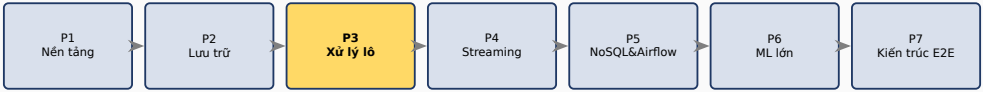
Câu hỏi ôn tập

1. Vì sao nhân bản 3 bản sao thay vì chỉ 1 hoặc 2?
2. Storage-compute separation giải quyết vấn đề gì so với HDFS truyền thống?
3. Time travel trong Lakehouse có thể hữu ích trong tình huống nào ở thực tế?

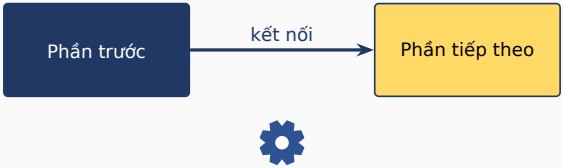
PHẦN 3

XỬ LÝ DỮ LIỆU THEO LÔ: MAPREDUCE & SPARK

Hai mô hình lập trình phân tán làm nền tảng tư duy xử lý Big Data



Kết nối từ Phần 2



Dữ liệu đã có nơi lưu trữ đáng tin cậy trên HDFS/Cloud/Lakehouse. Câu hỏi tiếp theo:

- Làm sao viết một chương trình chạy **song song** trên hàng nghìn máy để xử lý dữ liệu đó?

Mục tiêu học tập của Phần 3

- Hiểu mô hình lập trình MapReduce và quy trình phát triển ứng dụng bằng Java
- Hiểu kiến trúc Spark, khái niệm RDD, DataFrame và Spark SQL
- Phân tích được ưu điểm/hạn chế của từng mô hình và biết khi nào nên dùng công nghệ nào

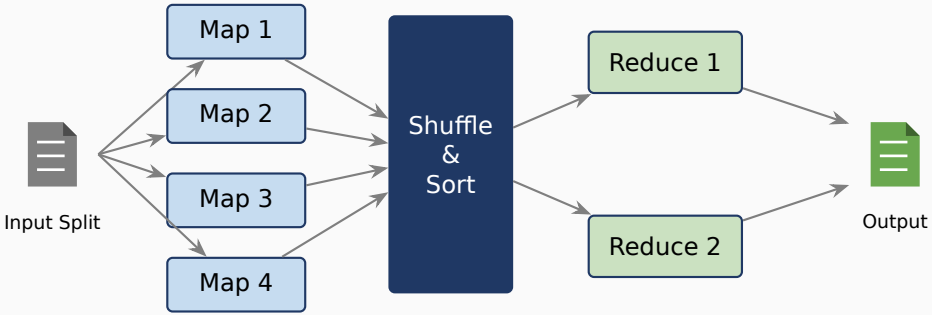
Bài toán: tính toán song song trên dữ liệu phân tán

- Dữ liệu đã được chia thành các block nằm rải rác trên nhiều DataNode (Phần 2)
- Thay vì kéo hết dữ liệu về một máy để xử lý (tốn băng thông, chậm), ý tưởng cốt lõi là: **đưa chương trình đến nơi dữ liệu đang nằm** (data locality)
- Đây chính là triết lý thiết kế của MapReduce

Mô hình MapReduce — ý tưởng

- Công bố bởi Dean & Ghemawat (Google, OSDI 2004) — lấy cảm hứng từ các hàm `map`/`reduce` trong lập trình hàm
- Chia bài toán thành hai giai đoạn đơn giản: **Map** (biến đổi từng phần tử độc lập) và **Reduce** (tổng hợp kết quả theo khóa)
- Lập trình viên chỉ cần viết logic Map/Reduce — framework Hadoop tự lo việc phân phối, chịu lỗi, và tổng hợp

Luồng xử lý MapReduce



Input được chia nhỏ (split) cho nhiều Map task xử lý song song; Shuffle & Sort gom nhóm theo khóa; Reduce tổng hợp kết quả cuối

Ví dụ WordCount — ý tưởng Map/Reduce

```
// Map: nhận một dòng văn bản, phát ra (từ, 1) cho mỗi từ
map(key, line):
    for word in line.split():
        emit(word, 1)

// Reduce: nhận (từ, [1,1,1,...]) đã gom nhóm, cộng dồn
reduce(word, counts):
    total = sum(counts)
    emit(word, total)
```

Ưu điểm của MapReduce

- Mô hình lập trình đơn giản: chỉ cần viết hai hàm Map và Reduce
- Tự động chịu lỗi: task thất bại được chạy lại trên máy khác mà không ảnh hưởng toàn bộ job
- Mở rộng tuyến tính: thêm máy vào cluster giúp xử lý nhanh hơn gần như tỉ lệ thuận

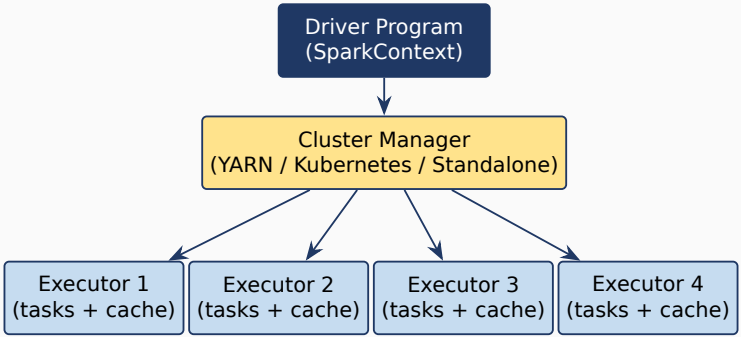
Hạn chế của MapReduce

- Kết quả trung gian giữa các giai đoạn Map/Reduce được ghi xuống **đĩa** (không phải RAM) $\Rightarrow$ độ trễ I/O cao
- Không phù hợp với các thuật toán **lặp** (iterative) như nhiều thuật toán học máy — mỗi vòng lặp lại phải đọc/ghi đĩa
- Không hỗ trợ tốt truy vấn tương tác (interactive query) — mỗi job đều có độ trễ khởi động đáng kể

Vì sao cần Spark?

- Ra đời tại AMPLab, UC Berkeley (2009), do Matei Zaharia và cộng sự phát triển, nhằm khắc phục hạn chế I/O đĩa của MapReduce
- Ý tưởng cốt lõi: giữ dữ liệu trung gian trong **bộ nhớ (RAM)** giữa các bước xử lý thay vì ghi xuống đĩa
- Kết quả: nhanh hơn MapReduce hàng chục lần với các workload lặp (machine learning, truy vấn tương tác)

Kiến trúc Apache Spark



Driver điều phối job; Cluster Manager (YARN/Kubernetes/Standalone) cấp phát tài nguyên; Executor thực thi task và cache dữ liệu

Driver, Executor, Cluster Manager

Driver Program

- Chạy hàm `main()`, tạo SparkContext
- Xây dựng kế hoạch thực thi (DAG) và điều phối các task

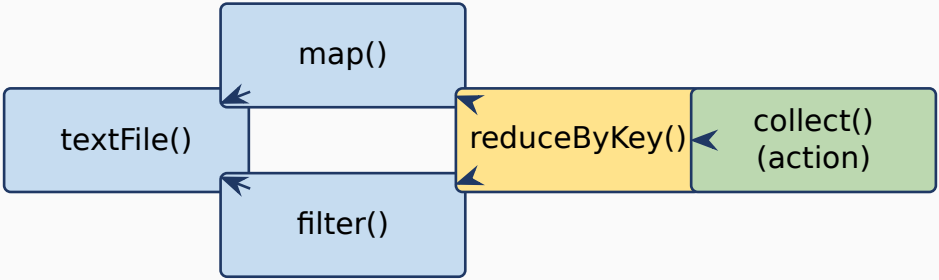
Executor

- Tiến trình chạy trên các worker node
- Thực thi task và **cache** dữ liệu trong RAM giữa các bước

RDD — Resilient Distributed Dataset

- Cấu trúc dữ liệu nền tảng của Spark: một tập hợp phần tử được phân vùng (partition) và phân tán trên các Executor
- **Resilient** (chịu lỗi): Spark ghi nhớ "lineage" (chuỗi phép biến đổi tạo ra RDD) để tự tính toán lại phần dữ liệu bị mất, không cần nhân bản như HDFS
- RDD là bất biến (immutable) — mọi phép biến đổi tạo ra một RDD mới

RDD DAG & Lazy Evaluation



Spark chỉ thực sự thực thi khi gặp một Action — cho phép tối ưu hóa toàn bộ chuỗi Transformation trước khi chạy

Transformation vs. Action

Loại	Đặc điểm	Ví dụ
Transformation	Lazy (chưa thực thi ngay), trả về RDD mới	map, filter, reduceByKey, join
Action	Kích hoạt thực thi toàn bộ DAG, trả kết quả cụ thể	collect, count, saveAsTextFile

Từ RDD đến DataFrame/Dataset

- RDD linh hoạt nhưng **không biết** cấu trúc dữ liệu bên trong — Spark không thể tối ưu truy vấn
- DataFrame: RDD có **schema** (tên cột, kiểu dữ liệu) — giống một bảng trong CSDL quan hệ
- Dataset (Scala/Java): DataFrame có kiểm tra kiểu dữ liệu tại thời điểm biên dịch (type-safe)

Spark SQL & Catalyst Optimizer

- Spark SQL cho phép truy vấn DataFrame bằng cú pháp SQL quen thuộc
- **Catalyst Optimizer**: tự động phân tích và tối ưu kế hoạch thực thi truy vấn (ví dụ: đẩy điều kiện lọc xuống sớm — predicate pushdown)
- Nhờ vậy DataFrame/Spark SQL thường nhanh hơn RDD thuần dù viết code đơn giản hơn

So sánh khái niệm: MapReduce vs. Spark

Tiêu chí	MapReduce	Spark
Lưu trung gian	Đĩa (disk)	Bộ nhớ (RAM), có thể tràn ra đĩa
Tốc độ (workload lặp)	Chậm	Nhanh hơn nhiều lần
API	Map/Reduce (Java)	RDD, DataFrame, SQL (đa ngôn ngữ)
Phù hợp	ETL đơn giản, một lượt	ETL, ML, truy vấn tương tác, streaming

Vị trí của Phần 3 trong Roadmap

- Phần 3 là **công cụ xử lý trung tâm** — Spark sẽ xuất hiện lại ở hầu hết các phần sau: Streaming (P4), MLlib (P6), và là thành phần chính trong kiến trúc end-to-end (P7)



Cầu nối sang Phần 4



*MapReduce và Spark (ở chế độ mặc định) xử lý dữ liệu **đã có sẵn** theo lô. Nhưng nhiều bài toán (Phần Ngữ cảnh & Ứng dụng) cần xử lý ngay khi dữ liệu phát sinh:*

- Gian lận thẻ tín dụng, cảnh báo IoT không thể chờ đến job batch tiếp theo
- $\rightarrow$ Phần 4 giới thiệu Kafka và Flink cho xử lý thời gian thực

Tổng kết Phần 3

Những điểm mấu chốt:

- MapReduce: mô hình Map–Shuffle–Reduce, đơn giản nhưng chậm vì ghi đĩa giữa các bước
- Spark: giữ dữ liệu trong RAM, nhanh hơn nhiều với workload lặp
- RDD là cấu trúc nền tảng; DataFrame/Spark SQL thêm schema và tối ưu hóa qua Catalyst
- Transformation là lazy; Action mới kích hoạt thực thi

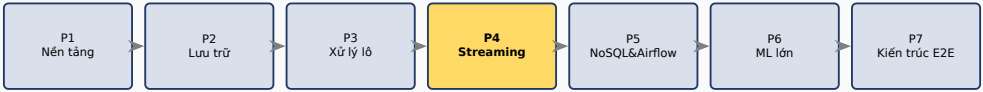
Câu hỏi ôn tập

1. Vì sao Spark nhanh hơn MapReduce với các thuật toán lặp như học máy?
2. Lazy evaluation mang lại lợi ích tối ưu hóa nào cho Spark?
3. Khi nào bạn vẫn có thể chọn MapReduce thay vì Spark?

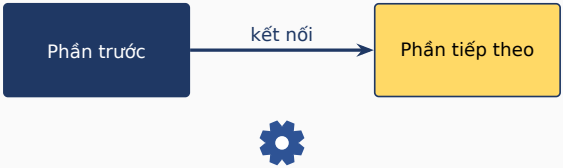
PHẦN 4

XỬ LÝ DỮ LIỆU THỜI GIAN THỰC

Apache Kafka, Spark Structured Streaming và Apache Flink



Kết nối từ Phần 3



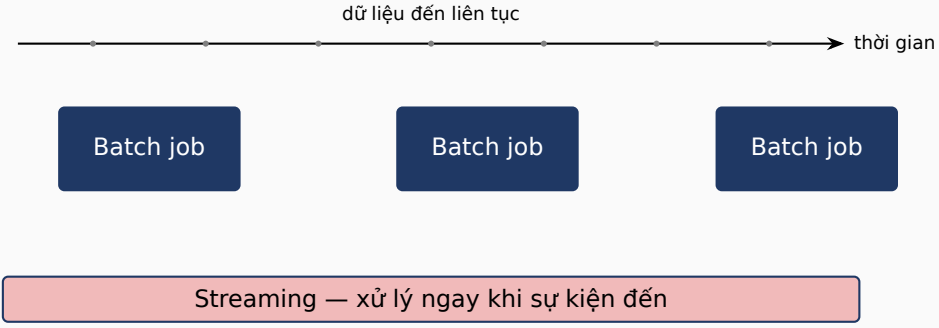
*Phần 3 giải quyết bài toán xử lý dữ liệu **đã có sẵn**. Nhưng nhớ lại ví dụ gian lận thẻ tín dụng và cảm biến IoT ở phần Ngữ cảnh:*

- Nhiều bài toán không thể chờ đến job batch tiếp theo — cần xử lý **ngay khi sự kiện xảy ra**

Mục tiêu học tập của Phần 4

- Phân biệt xử lý batch và streaming
- Hiểu kiến trúc Apache Kafka: producer, topic, partition, consumer
- Hiểu mô hình micro-batch (Spark Structured Streaming) và true streaming (Flink)
- Hiểu khái niệm event-time, windowing và dữ liệu đến trễ

Batch vs. Streaming



Batch xử lý theo chu kỳ (độ trễ phút–giờ); Streaming xử lý ngay khi sự kiện đến (độ trễ mili-giây–giây)

Khi nào cần Streaming?

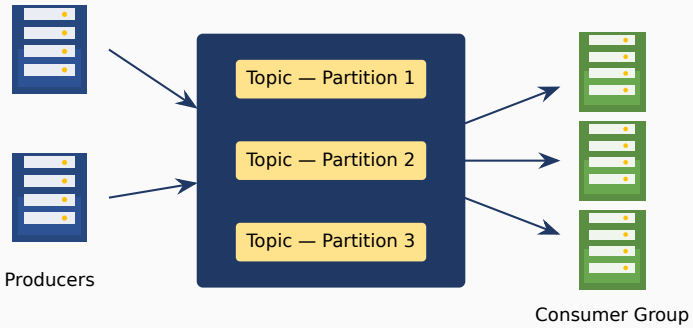
- Phát hiện gian lận giao dịch thẻ tín dụng **trước khi** giao dịch được chấp thuận
- Cảnh báo bất thường từ cảm biến IoT trong dây chuyền sản xuất hoặc lưới điện
- Cập nhật dashboard giám sát hệ thống theo thời gian thực
- Điểm chung: giá trị của thông tin **giảm mạnh** nếu xử lý chậm trễ



Apache Kafka là gì?

- Một nền tảng **log phân tán** (distributed log) đóng vai trò message broker giữa các hệ thống sinh dữ liệu và hệ thống xử lý
- Ban đầu phát triển tại LinkedIn (2011), hiện là một trong những hạ tầng streaming phổ biến nhất — hơn 150.000 tổ chức sử dụng theo các khảo sát công nghiệp gần đây
- Vai trò trung tâm: tách rời (decouple) hệ thống sinh dữ liệu và hệ thống tiêu thụ dữ liệu

Kiến trúc Kafka



Producer ghi message vào Topic (chia thành nhiều Partition); Consumer Group đọc song song từ các partition

Producer / Topic / Partition / Consumer Group

- **Producer**: ứng dụng ghi message vào Kafka (ví dụ: hệ thống giao dịch ghi log mỗi giao dịch)
- **Topic**: một "kênh" dữ liệu theo chủ đề (ví dụ: `transactions`)
- **Partition**: mỗi topic được chia thành nhiều partition để xử lý song song và mở rộng
- **Consumer Group**: nhóm các consumer cùng đọc một topic, mỗi partition chỉ được đọc bởi một consumer trong nhóm tại một thời điểm

Vì sao Kafka mở rộng tốt?

- Giống HDFS ở tầng lưu trữ, Kafka phân vùng (partition) dữ liệu để xử lý song song
- Mỗi partition được nhân bản trên nhiều broker để chịu lỗi — cùng triết lý với block replication của HDFS (Phần 2)
- Có thể thêm broker/partition để tăng thông lượng gần như tuyến tính

Ví dụ lệnh Kafka CLI

```
# Tạo một topic mới với 3 partition
kafka-topics.sh --create --topic transactions \
  --partitions 3 --replication-factor 2

# Gửi message (producer)
kafka-console-producer.sh --topic transactions

# Nhận message (consumer)
kafka-console-consumer.sh --topic transactions \
  --from-beginning
```

Spark Structured Streaming

- Mở rộng DataFrame API của Spark (Phần 3) để xử lý luồng dữ liệu liên tục
- Mô hình **micro-batch**: chia luồng dữ liệu thành các lô rất nhỏ (ví dụ mỗi 1 giây) và xử lý bằng engine Spark SQL quen thuộc
- Ưu điểm: tái sử dụng toàn bộ kiến thức DataFrame/Spark SQL đã học ở Phần 3

Ví dụ: đọc dữ liệu từ Kafka bằng Spark Structured Streaming

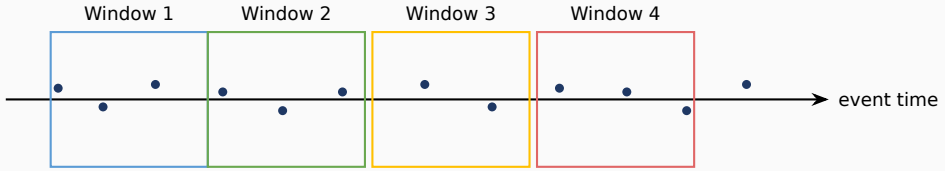
```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "transactions") \
    .load()

query = df.selectExpr("CAST(value AS STRING)") \
    .writeStream \
    .format("console") \
    .start()
```

Apache Flink — True Streaming

- Khác với micro-batch, Flink xử lý **từng sự kiện một** ngay khi đến — độ trễ thấp hơn nữa
- Được thiết kế từ đầu cho bài toán streaming (không phải mở rộng từ engine batch như Spark)
- Phù hợp các bài toán đòi hỏi độ trễ cực thấp và ngữ nghĩa thời gian phức tạp (event-time processing)

Event-time, Processing-time & Windowing



Flink nhóm sự kiện theo cửa sổ thời gian (window) dựa trên thời điểm sự kiện thực sự xảy ra (event-time), không phải thời điểm hệ thống nhận được nó

Khi nào dùng Kafka+Spark Streaming, khi nào dùng Flink?

Tiêu chí	Spark Structured Streaming	Apache Flink
Mô hình	Micro-batch	True streaming (theo từng sự kiện)
Độ trễ	Giây (thấp)	Mili-giây (rất thấp)
Hệ sinh thái	Liền mạch với Spark batch/MLlib	Mạnh về windowing/event-time nâng cao
Phù hợp khi	Đội ngũ đã dùng Spark, cần liên kết batch+stream	Yêu cầu độ trễ cực thấp, logic thời gian phức tạp

Watermark & dữ liệu đến trễ (Late Data)

- Trong hệ thống phân tán, sự kiện có thể đến **trễ** so với thời điểm nó thực sự xảy ra (ví dụ mất kết nối mạng tạm thời)
- **Watermark**: cơ chế đánh dấu "đến thời điểm này, ta xem như không còn dữ liệu cũ hơn X sẽ đến nữa", giúp hệ thống biết khi nào có thể chốt kết quả một window
- Đánh đổi giữa **độ trễ** (chờ dữ liệu trễ) và **độ đầy đủ** (accept dữ liệu đến muộn)

Thách thức của hệ thống Streaming

- **Exactly-once semantics**: đảm bảo mỗi sự kiện được xử lý đúng một lần, không bị đếm trùng hoặc bỏ sót khi có lỗi
- **Thứ tự sự kiện**: dữ liệu từ nhiều partition có thể đến không đúng thứ tự thời gian
- **Khả năng mở rộng**: hệ thống phải xử lý được các đợt tăng đột biến (traffic spike) mà không mất dữ liệu

Case study: Hệ thống phát hiện gian lận thời gian thực



- Producer: hệ thống thanh toán ghi mỗi giao dịch vào Kafka topic `transactions`
- Spark Structured Streaming / Flink: tính toán các đặc trưng (feature) theo cửa sổ thời gian (ví dụ: số giao dịch trong 5 phút gần nhất)
- Mô hình học máy (huấn luyện từ trước — sẽ học ở Phần 6) đánh giá điểm rủi ro theo thời gian thực
- Nếu điểm rủi ro vượt ngưỡng $\rightarrow$ chặn giao dịch trước khi hoàn tất

Vị trí của Phần 4 trong Roadmap

- Phần 4 bổ sung năng lực **Velocity** còn thiếu ở Phần 3
- Sẽ kết hợp với Lakehouse (Phần 2) trong kiến trúc end-to-end ở Phần 7 (ví dụ: Kafka → Spark Streaming → Delta Lake)



Cầu nối sang Phần 5



Dữ liệu streaming/batch giờ đã có thể lưu vào Lakehouse. Nhưng không phải mọi bài toán đều phù hợp với lưu trữ dạng bảng:

- Dữ liệu quan hệ mạng xã hội cần cơ sở dữ liệu đồ thị; dữ liệu phiên làm việc cần key-value tốc độ cao
- Và khi hệ thống có nhiều bước xử lý nối tiếp nhau, ai sẽ điều phối chúng chạy đúng thứ tự, đúng lịch?
- → Phần 5 giới thiệu hệ sinh thái NoSQL đa mô hình và Apache Airflow

Tổng kết Phần 4

Những điểm mấu chốt:

- Streaming xử lý dữ liệu ngay khi phát sinh, độ trễ thấp hơn nhiều so với batch
- Kafka: nền tảng log phân tán với Producer, Topic, Partition, Consumer Group
- Spark Structured Streaming (micro-batch) và Flink (true streaming) là hai lựa chọn engine xử lý
- Watermark giải quyết bài toán dữ liệu đến trễ trong windowing

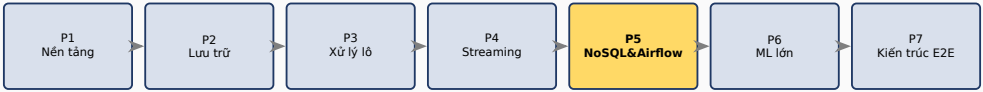
Câu hỏi ôn tập

1. Vì sao Kafka phân vùng (partition) dữ liệu, tương tự khái niệm nào đã học ở Phần 2?
2. Trong tình huống nào bạn sẽ ưu tiên Flink thay vì Spark Structured Streaming?
3. Watermark đánh đổi điều gì để xử lý dữ liệu đến trễ?

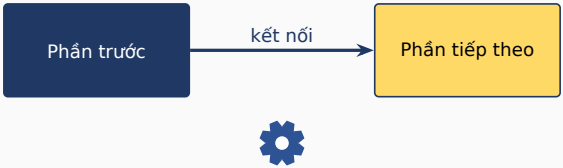
PHẦN 5

NOSQL ĐA MÔ HÌNH & ĐIỀU PHỐI PIPELINE

Các mô hình lưu trữ phi quan hệ và Apache Airflow



Kết nối từ Phần 4



Phần 2–4 đã trang bị: lưu trữ Lakehouse, xử lý batch và streaming. Nhưng không phải mọi dữ liệu đều hợp với mô hình bảng:

- Nhớ lại ví dụ mạng xã hội (dữ liệu đồ thị quan hệ) ở phần Ngữ cảnh — Lakehouse dạng bảng không phải lựa chọn tối ưu

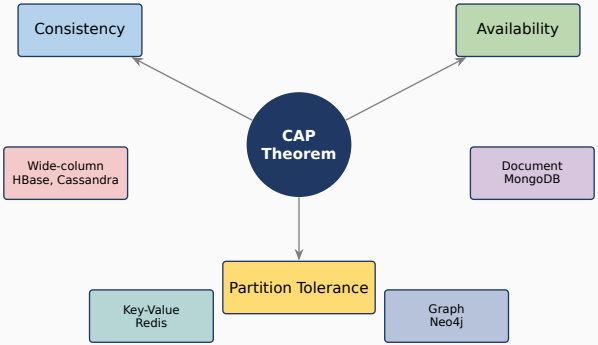
Mục tiêu học tập của Phần 5

- Hiểu định lý CAP và ý nghĩa đối với thiết kế hệ thống phân tán
- Phân biệt 4 loại NoSQL: wide-column, document, key-value, graph
- Hiểu khái niệm DAG, Task, Scheduler trong Apache Airflow
- Biết cách định nghĩa một pipeline ETL nhiều bước bằng Airflow

Vì sao một loại cơ sở dữ liệu là không đủ?

- Mỗi bài toán có đặc điểm truy cập dữ liệu khác nhau: quan hệ phức tạp (mạng xã hội), truy vấn theo khóa cực nhanh (session người dùng), hay ghi khối lượng cực lớn theo thời gian (log cảm biến)
- Không có một mô hình cơ sở dữ liệu nào tối ưu cho **mọi** trường hợp — đây là động lực ra đời phong trào NoSQL từ cuối những năm 2000

Định lý CAP



Trong một hệ phân tán, khi xảy ra phân vùng mạng (Partition), hệ thống phải đánh đổi giữa Consistency và Availability

Giải thích Consistency, Availability, Partition Tolerance

- **Consistency**: mọi lần đọc đều trả về dữ liệu mới nhất đã ghi (giống nhau ở mọi node)
- **Availability**: hệ thống luôn phản hồi yêu cầu, kể cả khi một phần cluster gặp sự cố
- **Partition Tolerance**: hệ thống vẫn hoạt động dù mất kết nối mạng giữa các node — **bắt buộc phải có** trong hệ phân tán thực tế
- $\Rightarrow$ Thực tế là lựa chọn giữa **CP** (ưu tiên nhất quán) hay **AP** (ưu tiên sẵn sàng)

Wide-column Store: HBase, Cassandra

- Dữ liệu tổ chức theo hàng (row key) và nhóm cột (column family) — mỗi hàng có thể có số cột khác nhau
- Tối ưu cho ghi khối lượng lớn, truy vấn theo row key rất nhanh
- HBase chạy trên nền HDFS (liên hệ Phần 2); Cassandra là hệ thống độc lập, phân tán ngang hàng (peer-to-peer)
- Use case: lưu trữ log sự kiện, dữ liệu chuỗi thời gian (time-series) quy mô lớn

Document Store: MongoDB

- Lưu dữ liệu dạng tài liệu (document), thường là JSON/BSON, không cần schema cố định trước
- Linh hoạt khi cấu trúc dữ liệu thay đổi thường xuyên hoặc không đồng nhất giữa các bản ghi
- Use case: hồ sơ sản phẩm thương mại điện tử với thuộc tính đa dạng theo từng ngành hàng

Key-Value Store: Redis

- Mô hình đơn giản nhất: mỗi giá trị được truy cập qua một khóa duy nhất, thường lưu toàn bộ trong RAM
- Tốc độ đọc/ghi cực nhanh (dưới mili-giây)
- Use case: cache kết quả truy vấn, lưu phiên đăng nhập (session), bộ đếm thời gian thực

Graph Database: Neo4j

- Dữ liệu tổ chức thành **node** (thực thể) và **edge** (quan hệ) — tự nhiên cho dữ liệu có tính kết nối cao
- Truy vấn quan hệ nhiều bước (ví dụ: "bạn của bạn") hiệu quả hơn nhiều so với JOIN nhiều bảng trong SQL
- Use case: mạng xã hội, hệ thống gợi ý dựa trên quan hệ, phát hiện gian lận qua mạng lưới giao dịch



So sánh 4 loại NoSQL — khi nào dùng loại nào?

Loại	Đại diện	Khi nào dùng
Wide-column	HBase, Cassandra	Ghi khối lượng lớn, dữ liệu time-series
Document	MongoDB	Schema linh hoạt, dữ liệu bán cấu trúc
Key-Value	Redis	Cache, session, độ trễ đọc/ghi cực thấp
Graph	Neo4j	Dữ liệu có quan hệ phức tạp, nhiều bước liên kết

Polyglot Persistence

- Một hệ thống thực tế thường **kết hợp nhiều loại** cơ sở dữ liệu, mỗi loại cho một mục đích phù hợp nhất — gọi là "polyglot persistence"
- Ví dụ: Redis cho cache phiên, MongoDB cho hồ sơ sản phẩm, Lakehouse (Phần 2) cho phân tích lịch sử
- Hệ quả: cần một công cụ điều phối các bước dữ liệu di chuyển **giữa** các hệ thống này

Vấn đề: điều phối pipeline nhiều bước

- Một pipeline ETL thực tế thường có nhiều bước phụ thuộc nhau: trích xuất → kiểm tra chất lượng → làm sạch → nạp vào kho dữ liệu → thông báo
- Cần chạy đúng **thứ tự**, đúng **lịch**, tự động **thử lại** khi lỗi, và có khả năng giám sát
- Làm thủ công (cron job rời rạc) rất khó bảo trì khi số lượng pipeline tăng lên

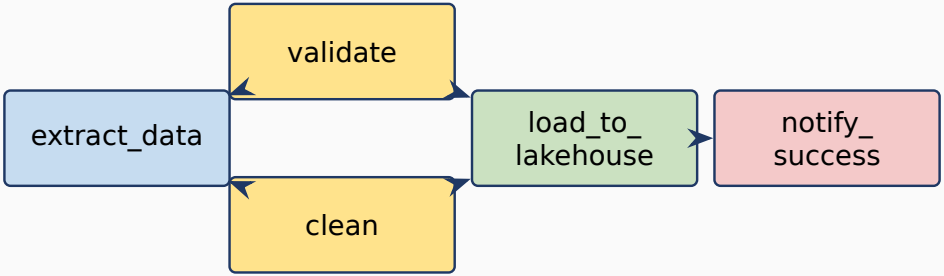
Apache Airflow là gì?

- Một nền tảng điều phối workflow (workflow orchestrator) mã nguồn mở, ban đầu phát triển tại Airbnb (2014)
- Cho phép định nghĩa pipeline bằng **code Python**, dễ kiểm soát phiên bản (version control) và tái sử dụng
- Cung cấp giao diện giám sát trực quan: trạng thái từng bước, lịch sử chạy, log lỗi

DAG, Task, Operator trong Airflow

- **DAG** (Directed Acyclic Graph): toàn bộ pipeline, định nghĩa các bước và thứ tự phụ thuộc
- **Task**: một bước cụ thể trong DAG (ví dụ: `extract_data`)
- **Operator**: loại hành động một Task thực hiện (chạy Python, chạy lệnh Bash, gọi Spark job...)

Ví dụ DAG trong Airflow



extract_data chạy trước; validate và clean chạy song song sau đó; load_to_lakehouse chờ cả hai hoàn tất trước khi notify_success

Ví dụ định nghĩa DAG bằng Python

```
from airflow import DAG
from airflow.operators.python import PythonOperator

with DAG("etl_pipeline", schedule="@daily") as dag:
    extract = PythonOperator(
        task_id="extract_data", python_callable=extract_fn)
    load = PythonOperator(
        task_id="load_to_lakehouse", python_callable=load_fn)

    extract >> load  # định nghĩa thứ tự phụ thuộc
```

Scheduler & Monitoring trong vận hành thực tế

- **Scheduler**: quyết định khi nào một DAG/Task được kích hoạt, theo lịch (cron) hoặc theo sự kiện
- **Retry & Alerting**: tự động thử lại Task thất bại theo cấu hình; gửi cảnh báo (email/Slack) khi pipeline lỗi
- Airflow không **xử lý dữ liệu** — nó chỉ điều phối; công việc xử lý thực tế vẫn do Spark/Kafka (đã học) đảm nhiệm

Vị trí của Phần 5 trong Roadmap

- Phần 5 mở rộng khả năng lưu trữ (bên cạnh Lakehouse ở Phần 2) và bổ sung năng lực **điều phối** toàn bộ pipeline
- Airflow sẽ điều phối toàn bộ kiến trúc end-to-end ở Phần 7



Cầu nối sang Phần 6



Dữ liệu đã sẵn sàng: được lưu trữ đúng mô hình, và pipeline đã được điều phối tự động. Câu hỏi tiếp theo:

- Làm sao dùng toàn bộ hạ tầng này để **dự đoán** thay vì chỉ mô tả dữ liệu đã xảy ra?
- $\rightarrow$ Phần 6 giới thiệu học máy quy mô lớn với Spark MLlib

Tổng kết Phần 5

Những điểm mấu chốt:

- Định lý CAP: phải đánh đổi giữa Consistency và Availability khi có phân vùng mạng
- 4 loại NoSQL: wide-column, document, key-value, graph — mỗi loại phù hợp một dạng bài toán
- Polyglot persistence: kết hợp nhiều loại CSDL trong cùng một hệ thống
- Airflow điều phối pipeline qua DAG, Task, Operator, Scheduler

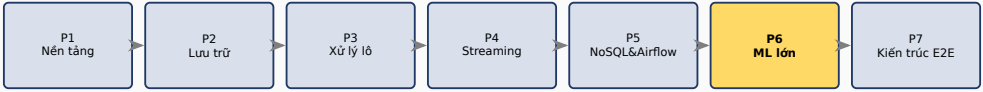
Câu hỏi ôn tập

1. Vì sao mạng xã hội phù hợp với graph database hơn là bảng quan hệ truyền thống?
2. Nêu một ví dụ polyglot persistence khác ngoài ví dụ đã học
3. Vì sao cần Scheduler tự động thay vì chạy tay từng bước ETL?

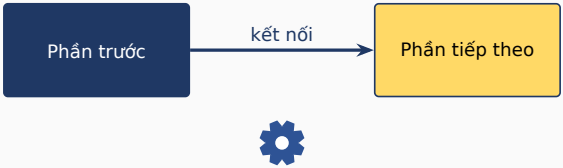
PHẦN 6

HỌC MÁY QUY MÔ LỚN TRÊN BIG DATA

Spark MLlib, Feature Store và kết nối với Deep Learning phân tán



Kết nối từ Phần 5



Dữ liệu đã sẵn sàng: lưu trữ đúng mô hình (P2, P5), xử lý được cả batch lẫn streaming (P3, P4), pipeline được điều phối tự động (P5). Giờ là lúc dùng toàn bộ hạ tầng đó để:

- Không chỉ **mô tả** dữ liệu đã xảy ra, mà **dự đoán** điều sắp xảy ra

Mục tiêu học tập của Phần 6

- Hiểu vì sao cần công cụ học máy chuyên biệt cho quy mô Big Data
- Nắm được Pipeline API của Spark MLlib và các nhóm thuật toán chính
- Hiểu khái niệm Feature Store và vai trò trong MLOps
- Biết cách Big Data kết nối với huấn luyện deep learning phân tán

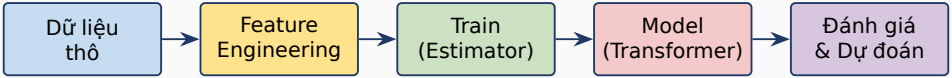
Vì sao học máy truyền thống không đủ ở quy mô Big Data?

- Thư viện quen thuộc như scikit-learn chạy trên **một máy**, giả định dữ liệu vừa trong RAM
- Với dữ liệu hàng trăm GB–TB (ví dụ log hành vi TMĐT ở phần Ngữ cảnh), cần thuật toán học máy chạy **phân tán** trên cùng hạ tầng Spark đã học ở Phần 3
- Đây chính là lý do ra đời Spark MLlib — thư viện học máy tích hợp sẵn trong Spark

Spark MLlib là gì?

- Thư viện học máy phân tán, xây dựng trên cùng nền RDD/DataFrame đã học ở Phần 3
- Tận dụng toàn bộ hạ tầng xử lý phân tán, cache trong RAM, và khả năng mở rộng của Spark
- Cung cấp cả thuật toán học máy cổ điển lẫn công cụ tiền xử lý đặc trưng (feature engineering)

Pipeline API: Transformer & Estimator



Mỗi bước xử lý (feature engineering, huấn luyện) được ghép nối thành một Pipeline duy nhất, dễ tái sử dụng và triển khai

Nhóm thuật toán 1 — Hồi quy

- Linear Regression, Logistic Regression — mở rộng trực tiếp từ kỹ thuật Regression đã học ở Phần 1, nay chạy phân tán trên hàng triệu bản ghi
- Use case: dự đoán giá trị liên tục (doanh thu) hoặc xác suất nhị phân (khách hàng có rời bỏ hay không)

Nhóm thuật toán 2 — Phân loại

- Decision Tree, Random Forest, Gradient-Boosted Trees — mở rộng khái niệm Classification đã học ở Phần 1
- Random Forest: kết hợp nhiều cây quyết định để tăng độ chính xác và giảm overfitting
- Use case: phát hiện gian lận (mở rộng từ Phần 4), phân loại khách hàng tiềm năng

Nhóm thuật toán 3 — Phân cụm

- K-Means: mở rộng khái niệm Clustering đã học ở Phần 1, chạy phân tán trên tập dữ liệu lớn
- Use case: phân khúc khách hàng ở quy mô hàng triệu người dùng cho một sàn TMĐT lớn

Ví dụ: xây dựng Pipeline MLlib

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import RandomForestClassifier

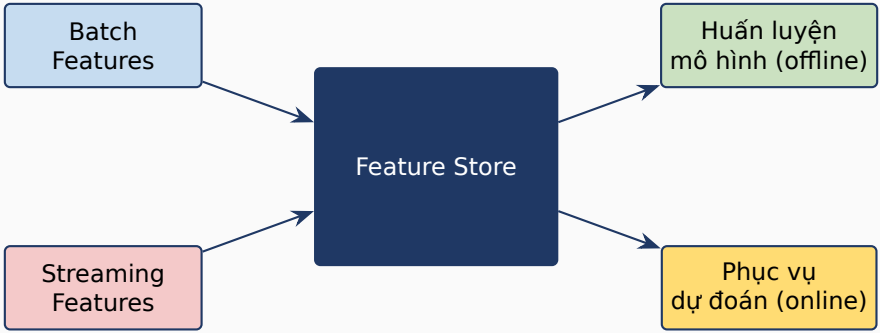
assembler = VectorAssembler(
    inputCols=["age", "income", "clicks"], outputCol="features")
rf = RandomForestClassifier(labelCol="churn", featuresCol="features")

pipeline = Pipeline(stages=[assembler, rf])
model = pipeline.fit(training_data)
predictions = model.transform(test_data)
```

Đánh giá mô hình ở quy mô lớn

- Chia tập train/test cần thực hiện **phân tán**, không thể gom hết dữ liệu về một máy để chia như scikit-learn
- Cross-validation trong MLlib chạy song song nhiều fold trên cluster, tăng tốc đáng kể quá trình chọn tham số (hyperparameter tuning)
- Các độ đo đánh giá (accuracy, F1, AUC...) cũng được tính toán phân tán trên toàn bộ tập dữ liệu

Feature Store là gì?



Feature Store là nơi tập trung lưu trữ và phục vụ đặc trưng (feature), đảm bảo tính nhất quán giữa lúc huấn luyện (offline) và lúc dự đoán (online)

Feature Offline vs. Online

- **Offline (Batch Features)**: đặc trưng tính từ dữ liệu lịch sử trong Lakehouse (Phần 2), dùng để huấn luyện mô hình
- **Online (Streaming Features)**: đặc trưng tính gần thời gian thực từ Kafka/Flink (Phần 4), dùng khi phục vụ dự đoán (serving)
- Vấn đề "training-serving skew": nếu hai đường tính đặc trưng không nhất quán, mô hình hoạt động khác nhau giữa huấn luyện và thực tế — Feature Store giải quyết vấn đề này

Giới thiệu Horovod

- Framework mã nguồn mở (Uber, 2017) để huấn luyện mô hình deep learning **phân tán** trên nhiều máy/GPU
- Hoạt động cùng TensorFlow, PyTorch — có thể chạy trên cùng cluster Spark đã có sẵn
- Dùng thuật toán "ring all-reduce" để đồng bộ gradient hiệu quả giữa các worker

Giới thiệu Ray

- Framework tính toán phân tán hiện đại cho Python, phát triển tại UC Berkeley RISELab — cùng nơi khai sinh Spark
- Được thiết kế linh hoạt cho cả huấn luyện học máy, tuning siêu tham số, và phục vụ mô hình (model serving)
- Ngày càng phổ biến trong hệ sinh thái AI hiện đại như một lựa chọn thay thế/bổ sung cho Spark ở các workload AI chuyên sâu

PyTorch Distributed & kết nối với Big Data

- `torch.distributed`: module gốc của PyTorch để huấn luyện mô hình song song trên nhiều GPU/máy
- Trong thực tế: dữ liệu huấn luyện thường được chuẩn bị bằng Spark (làm sạch, trích đặc trưng ở quy mô lớn), sau đó nạp vào pipeline huấn luyện PyTorch Distributed
- Đây là điểm giao thoa trực tiếp giữa hạ tầng Big Data (đã học suốt Phần 1–5) và các kỹ thuật deep learning hiện đại

MLOps là gì?

- Tập hợp thực hành đưa mô hình học máy vào vận hành thực tế một cách tin cậy, có thể lặp lại và giám sát được
- Vòng đời: chuẩn bị dữ liệu (Phần 2, 5) $\rightarrow$ huấn luyện (MLlib/Horovod/PyTorch) $\rightarrow$ đóng gói $\rightarrow$ triển khai $\rightarrow$ giám sát hiệu năng $\rightarrow$ huấn luyện lại (retrain)
- Feature Store, Airflow (Phần 5), và Lakehouse (Phần 2) đều là các mảnh ghép hạ tầng phục vụ MLOps

Case study: Hệ thống gợi ý sản phẩm quy mô lớn



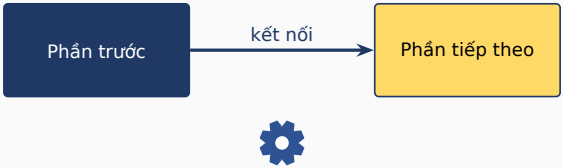
- Dữ liệu hành vi người dùng được thu thập qua Kafka (Phần 4) và lưu vào Lakehouse (Phần 2)
- Spark MLlib huấn luyện định kỳ mô hình gợi ý trên toàn bộ lịch sử (offline)
- Feature Store phục vụ đặc trưng thời gian thực khi người dùng đang duyệt web (online)
- Airflow (Phần 5) điều phối toàn bộ chu trình huấn luyện lại định kỳ

Vị trí của Phần 6 trong Roadmap

- Phần 6 là **đỉnh cao ứng dụng** của toàn bộ hạ tầng đã xây dựng từ Phần 1 đến Phần 5
- Đây cũng là phần kết nối trực tiếp Big Data với các kỹ thuật AI/Deep Learning hiện đại



Cầu nối sang Phần 7



Chúng ta đã có đầy đủ mọi mảnh ghép: lưu trữ, xử lý batch/streaming, NoSQL, điều phối, và học máy quy mô lớn.

- Câu hỏi cuối cùng: làm sao **ghép** tất cả các mảnh ghép này thành MỘT kiến trúc hệ thống hoàn chỉnh, vận hành được trong thực tế?
- $\rightarrow$ Phần 7 — chương tổng hợp và dự án capstone

Tổng kết Phần 6

Những điểm mấu chốt:

- Spark MLlib mở rộng các kỹ thuật data mining (Phần 1) lên quy mô phân tán
- Pipeline API (Transformer/Estimator) chuẩn hóa quy trình từ tiền xử lý đến huấn luyện
- Feature Store đảm bảo nhất quán giữa huấn luyện (offline) và phục vụ (online)
- Horovod, Ray, PyTorch Distributed là cầu nối giữa Big Data và Deep Learning hiện đại

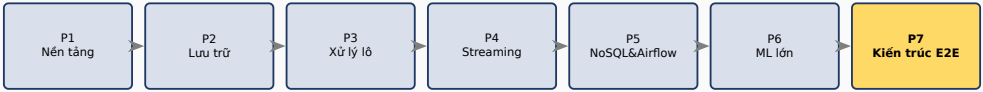
Câu hỏi ôn tập

1. Vì sao cần chia train/test theo cách phân tán thay vì gom về một máy?
2. Feature Store giải quyết vấn đề "training-serving skew" như thế nào?
3. Trong dự án capstone của nhóm bạn, phần nào sẽ dùng Spark MLlib?

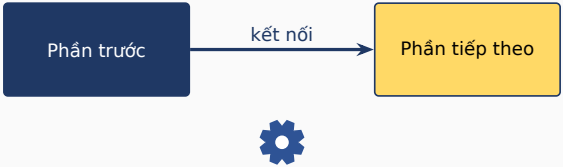
PHẦN 7

KIẾN TRÚC ỨNG DỤNG BIG DATA END-TO-END

Tổng hợp toàn bộ roadmap thành một hệ thống hoàn chỉnh — Dự án Capstone



Kết nối từ Phần 6 — nhìn lại cả hành trình



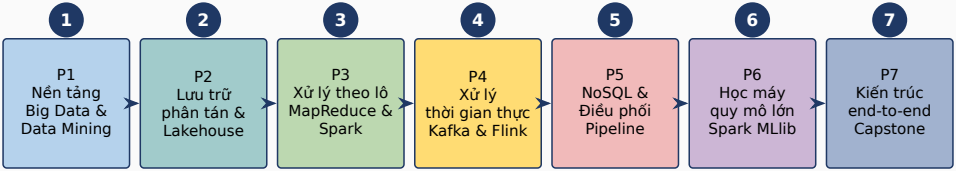
Sáu phần trước đã lần lượt trang bị từng mảnh ghép kỹ thuật:

- P1: khái niệm & kỹ thuật khai phá dữ liệu — P2: lưu trữ phân tán & Lakehouse
- P3: xử lý theo lô (MapReduce/Spark) — P4: xử lý thời gian thực (Kafka/Flink)
- P5: NoSQL & điều phối pipeline (Airflow) — P6: học máy quy mô lớn (MLlib)
- Phần 7 không giới thiệu công nghệ mới — mà dạy cách **phối hợp** tất cả

Mục tiêu học tập của Phần 7

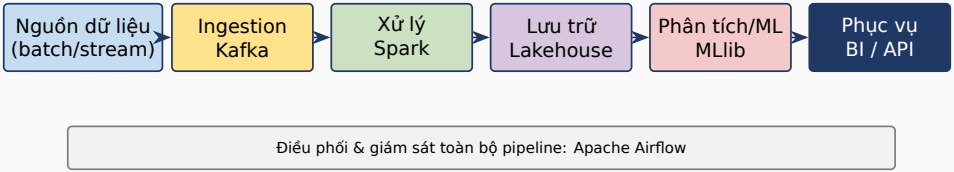
- Nắm được nguyên tắc thiết kế kiến trúc Big Data end-to-end
- Phân biệt được Lambda Architecture và Kappa Architecture
- Biết cách lựa chọn công nghệ phù hợp theo đặc điểm bài toán thực tế
- Hiểu rõ yêu cầu và tiêu chí đánh giá của dự án capstone (Tuần 8)

Nhắc lại toàn cảnh Roadmap



Phần 7 là điểm hội tụ của toàn bộ 6 phần trước — không phải một mắt xích rời rạc

Nguyên tắc thiết kế kiến trúc End-to-End



Bốn tầng cốt lõi: Ingestion → Xử lý → Lưu trữ → Phân tích/Phục vụ, được điều phối xuyên suốt bởi Airflow

Tầng Ingestion — Batch hay Streaming?

- Nếu dữ liệu chỉ cần cập nhật định kỳ (báo cáo hàng ngày) → batch ingestion đơn giản, chi phí thấp hơn
- Nếu bài toán đòi hỏi phản hồi tức thời (gian lận, cảnh báo) → bắt buộc dùng Kafka (Phần 4)
- Nhiều hệ thống thực tế dùng **cả hai** cho các mục đích khác nhau — đây là tiền đề của Lambda Architecture

Tầng Lưu trữ — Lakehouse hay NoSQL?

- Dữ liệu phân tích dạng bảng, cần ACID và truy vấn SQL → Lakehouse (Delta Lake/Iceberg, Phần 2)
- Dữ liệu quan hệ phức tạp, session, cache tốc độ cao → NoSQL phù hợp (Phần 5)
- Một hệ thống lớn thường dùng **polyglot persistence**: Lakehouse cho phân tích lịch sử, NoSQL cho truy vấn vận hành thời gian thực

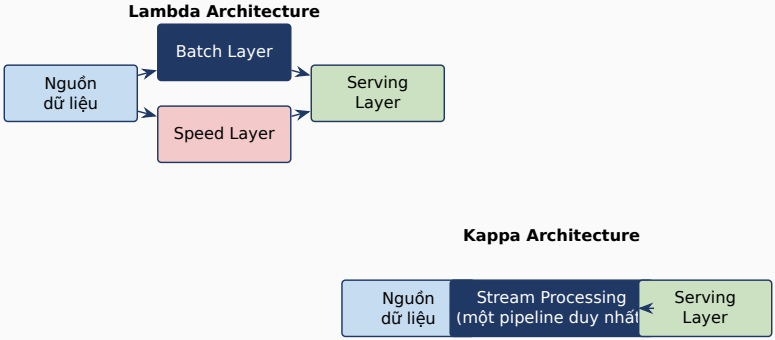
Tầng Xử lý — Spark Batch, Streaming, hay cả hai?

- Spark (Phần 3) xử lý cả batch (DataFrame/SQL) và streaming (Structured Streaming, Phần 4) trên cùng một engine — lợi thế lớn khi đội ngũ đã quen thuộc với Spark
- MLlib (Phần 6) có thể chạy trực tiếp trên kết quả của tầng xử lý này để tạo ra dự đoán

Tầng Phục vụ — BI, API, hay ML Serving?

- **BI Dashboard**: truy vấn SQL trên Lakehouse phục vụ báo cáo, trực quan hóa cho người ra quyết định
- **API**: phục vụ dữ liệu/dự đoán cho ứng dụng khác (ví dụ ứng dụng di động) theo thời gian thực
- **ML Serving**: phục vụ mô hình học máy đã huấn luyện (Phần 6) để dự đoán trực tuyến

Lambda Architecture



Lambda: Batch Layer (chính xác, độ trễ cao) chạy song song với Speed Layer (gần đúng, độ trễ thấp), hợp nhất tại Serving Layer

Kappa Architecture — đơn giản hóa

- Nhận xét: duy trì **hai** pipeline riêng biệt (batch và speed) trong Lambda gây trùng lặp logic, khó bảo trì
- Kappa Architecture: chỉ dùng **một** pipeline streaming duy nhất — kể cả khi cần "tính lại từ đầu" (reprocessing), chỉ cần replay lại dữ liệu từ Kafka (log vẫn còn lưu)
- Khả thi nhờ Kafka lưu trữ log bền vững (Phần 4) — không cần một Batch Layer riêng

Lambda vs. Kappa — khi nào chọn kiến trúc nào?

Tiêu chí	Lambda	Kappa
Độ phức tạp	Cao (2 pipeline song song)	Thấp hơn (1 pipeline duy nhất)
Độ chính xác batch	Rất cao (tính lại toàn bộ)	Phụ thuộc khả năng replay stream
Phù hợp khi	Cần độ chính xác batch tuyệt đối, đội ngũ lớn	Ưu tiên đơn giản hóa vận hành, dữ liệu chủ yếu streaming

Nguyên tắc lựa chọn công nghệ theo bài toán thực tế

- **Chi phí**: cluster tự quản (on-premise) vs. dịch vụ quản lý trên cloud (trả theo nhu cầu)
- **Độ trễ yêu cầu**: batch (phút–giờ) hay streaming (giây–mili-giây)?
- **Quy mô dữ liệu & tốc độ tăng trưởng**: hệ thống có cần mở rộng nhanh trong tương lai gần?
- **Năng lực đội ngũ**: công nghệ phức tạp nhất chưa chắc là lựa chọn tốt nhất nếu đội ngũ chưa đủ kinh nghiệm vận hành

Giới thiệu đề bài Dự án Capstone

- Làm việc theo nhóm, xây dựng một **pipeline Big Data end-to-end tối thiểu** áp dụng trên một bộ dữ liệu thực tế do nhóm tự chọn
- Đề tài nên xuất phát từ một trong các ứng dụng đã thảo luận ở đầu buổi học (TMĐT, tài chính, y tế, IoT...)
- Mục tiêu: minh chứng khả năng **tích hợp** nhiều công nghệ đã học, không chỉ dùng một công nghệ đơn lẻ

Yêu cầu kỹ thuật tối thiểu của Capstone

- Pipeline tham khảo: Kafka → Spark Structured Streaming → Delta Lake → MLlib/truy vấn
  - Ingestion: mô phỏng luồng dữ liệu qua Kafka (Phần 4)
  - Xử lý: Spark Structured Streaming biến đổi dữ liệu (Phần 3–4)
  - Lưu trữ: ghi vào bảng Lakehouse có ACID (Phần 2)
  - Phân tích: truy vấn SQL hoặc một mô hình MLlib đơn giản (Phần 6)
- Nhóm có thể thay thế một số thành phần nếu có lý do kỹ thuật hợp lý (ví dụ dùng Flink thay Spark Streaming)

Tiêu chí đánh giá Dự án Capstone (đề xuất)

Tiêu chí	Trọng số	Mô tả
Tính đúng đắn kỹ thuật	40%	Pipeline chạy được, đúng luồng dữ liệu end-to-end
Chiều sâu tích hợp	25%	Số lượng & mức độ hợp lý khi kết hợp các phần đã học
Chất lượng báo cáo	15%	Tài liệu kiến trúc rõ ràng, giải thích lựa chọn công nghệ
Thuyết trình nhóm	20%	Trình bày mạch lạc, trả lời được câu hỏi phản biện

Timeline làm việc nhóm cho Capstone

Thời điểm	Công việc
Tuần 2–3	Thành lập nhóm, chọn đề tài & bộ dữ liệu
Tuần 4–6	Xây dựng từng thành phần (lưu trữ, xử lý, streaming)
Tuần 7	Tích hợp pipeline, thêm thành phần MLlib
Tuần 8	Hoàn thiện, viết báo cáo, thuyết trình

Case study tham khảo: kiến trúc điển hình một nền tảng TMĐT lớn

- Ingestion: hành vi người dùng (click, xem, mua) đổ vào Kafka theo thời gian thực
- Xử lý: Spark Structured Streaming tính đặc trưng theo phiên (session); Spark batch tổng hợp báo cáo hàng ngày
- Lưu trữ: Lakehouse (Delta Lake) cho phân tích lịch sử; Redis cho cache gợi ý theo thời gian thực; MongoDB cho hồ sơ sản phẩm
- Phục vụ: MLlib/mô hình deep learning gợi ý sản phẩm; Airflow điều phối huấn luyện lại mô hình hàng đêm
- $\Rightarrow$ Đây là một ví dụ **minh họa mô hình kiến trúc điển hình**, không mô tả hệ thống cụ thể của bất kỳ công ty nào

Vị trí của Phần 7 trong Roadmap — đích đến cuối cùng

- Phần 7 không phải một mắt xích rời rạc, mà là nơi **tất cả** 6 phần trước hội tụ
- Hoàn thành Phần 7 nghĩa là đã có đủ năng lực thiết kế một hệ thống Big Data thực tế



Nhìn lại toàn bộ hành trình 7 phần

- P1 → P2 → P3: trục xương sống (khái niệm → lưu trữ → xử lý)
- P3 mở rộng song song thành P4 (streaming) và P5 (NoSQL/điều phối)
- P1–P5 hội tụ thành P6 (học máy quy mô lớn)
- Toàn bộ 6 phần hội tụ thành P7 (kiến trúc hoàn chỉnh) — đúng như sơ đồ roadmap đã thấy ở đầu buổi học

Tổng kết Phần 7 & chuẩn bị Tuần 9

Những điểm mấu chốt:

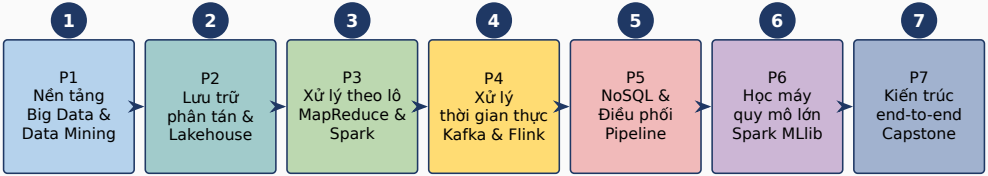
- Kiến trúc end-to-end gồm 4 tầng: Ingestion, Xử lý, Lưu trữ, Phục vụ, điều phối bởi Airflow
- Lambda Architecture: chính xác nhưng phức tạp (2 pipeline); Kappa: đơn giản hơn (1 pipeline streaming)
- Lựa chọn công nghệ luôn là bài toán đánh đổi: chi phí, độ trễ, quy mô, năng lực đội ngũ
- Dự án capstone là cơ hội thực hành tích hợp toàn bộ kiến thức của học phần

Câu hỏi ôn tập

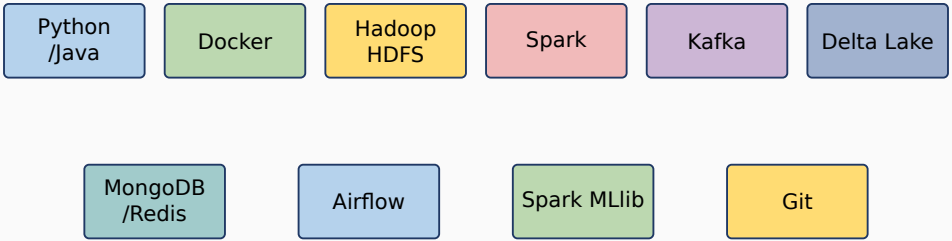
1. Vì sao Kappa Architecture khả thi được nhờ đặc tính nào của Kafka?
2. Với đề tài capstone nhóm bạn chọn, tầng nào là khó tích hợp nhất? Vì sao?
3. Ôn tập cho Tuần 9: hãy tự vẽ lại sơ đồ roadmap 7 phần từ trí nhớ

THIẾT LẬP MÔI TRƯỜNG THỰC HÀNH

Phần thực hành của Tuần 1 — chuẩn bị công cụ cho toàn bộ 9 tuần



Toàn cảnh công cụ sẽ sử dụng



Không cần cài đặt tất cả ngay hôm nay — một số công cụ (Kafka, Airflow...) sẽ cài ở đúng tuần cần dùng

Yêu cầu hệ thống khuyến nghị

- RAM: tối thiểu 8GB, khuyến nghị 16GB trở lên (chạy nhiều container/JVM cùng lúc)
- Dung lượng đĩa trống: tối thiểu 20GB
- Hệ điều hành: Linux hoặc macOS thuận lợi nhất; Windows nên dùng WSL2 (Windows Subsystem for Linux)
- Kết nối Internet ổn định để tải các gói phần mềm trong buổi thực hành hôm nay

Bước 1 — Cài đặt Python & môi trường ảo

```
# Kiểm tra phiên bản Python (khuyến nghị >= 3.9)
python3 --version

# Tạo môi trường ảo riêng cho môn học
python3 -m venv bigdata-env
source bigdata-env/bin/activate

# Cài các thư viện cơ bản sẽ dùng trong suốt khóa học
pip install pyspark pandas jupyter
```

Bước 2 — Cài đặt Java (JDK)

```
# Hadoop, Spark, Kafka đều chạy trên nền JVM — cần JDK 11 hoặc 17
sudo apt install openjdk-11-jdk

# Kiểm tra cài đặt
java -version

# Thiết lập biến môi trường JAVA_HOME
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
echo 'export JAVA_HOME=...' >> ~/.bashrc
```

Bước 3 — Vì sao dùng Docker cho môn học này?

- Docker đóng gói mỗi công cụ (Hadoop, Kafka, Airflow...) cùng toàn bộ phụ thuộc vào một **container** độc lập
- Tránh xung đột phiên bản giữa các công cụ, dễ dàng "dọn dẹp" và cài lại nếu có lỗi
- Giúp môi trường thực hành giống nhau trên mọi máy tính của sinh viên, giảm thiểu lỗi "chạy được trên máy tôi"

Bước 3 — Cài đặt Docker

```
# Cài Docker Engine (Ubuntu/Debian)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# Kiểm tra cài đặt
docker --version
docker run hello-world
```

Bước 4 — Cài đặt Git & thiết lập repository

```
# Cài Git
sudo apt install git

# Cấu hình thông tin cá nhân
git config --global user.name "Ho ten"
git config --global user.email "email@hcmute.edu.vn"

# Tạo repository cho bài tập môn học
mkdir bigdata-analytics-2026 && cd bigdata-analytics-2026
git init
```

Bước 5 — Cài đặt Hadoop (Pseudo-Distributed mode)

```
# Tải và giải nén Hadoop
wget https://downloads.apache.org/hadoop/common/.../hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz

# Định dạng HDFS lần đầu
hdfs namenode -format

# Khởi động HDFS & YARN
start-dfs.sh
start-yarn.sh
```

Bước 6 — Kiểm tra HDFS hoạt động

```
# Tạo thư mục làm việc trên HDFS
hdfs dfs -mkdir -p /user/bigdata

# Tải một file lên HDFS
hdfs dfs -put sample.txt /user/bigdata/

# Liệt kê nội dung
hdfs dfs -ls /user/bigdata/

# Giao diện web giám sát HDFS
# http://localhost:9870
```

Bước 7 — Cài đặt & kiểm tra Apache Spark (local mode)

```
# Tải và giải nén Spark
wget https://downloads.apache.org/spark/spark-3.5.0/spark-3.5.0-bin-hadoop3.tgz
tar -xzf spark-3.5.0-bin-hadoop3.tgz

# Chạy thử WordCount đầu tiên bằng PySpark shell
pyspark
>>> text = sc.textFile("sample.txt")
>>> counts = text.flatMap(lambda l: l.split()) \\
...     .map(lambda w: (w, 1)).reduceByKey(lambda a,b: a+b)
>>> counts.collect()
```

Các công cụ sẽ cài ở các tuần sau

- **Kafka**: cài đặt vào đầu Tuần 5 (Phần 4 — Xử lý thời gian thực)
- **MongoDB / Redis / Airflow**: cài đặt vào đầu Tuần 6 (Phần 5 — NoSQL & Điều phối Pipeline)
- **Delta Lake / Iceberg**: thêm thư viện vào Spark đã có sẵn, thực hiện ở Tuần 3 (Phần 2)
- Không cần lo lắng nếu chưa hiểu hết công dụng của các công cụ trên — mỗi công cụ sẽ được giải thích kỹ khi đến đúng tuần

Xử lý sự cố thường gặp khi cài đặt

- **Lỗi `JAVA_HOME not set`**: kiểm tra lại biến môi trường đã được export đúng đường dẫn JDK
- **Lỗi `Connection refused`** khi truy cập HDFS/Spark UI: kiểm tra dịch vụ đã `start-dfs.sh`/`start-yarn.sh` thành công chưa, và đúng cổng (port)
- **Lỗi quyền truy cập (Permission denied)**: kiểm tra quyền thư mục cài đặt, tránh cài ở thư mục hệ thống cần quyền root
- **Xung đột phiên bản Java**: nhiều JDK cùng cài trên máy — dùng `update-alternatives` để chọn đúng phiên bản

Checklist chuẩn bị trước Tuần 2

- ☐ Python 3.9+ và môi trường ảo đã hoạt động
- ☐ Java JDK 11/17 đã cài đặt, biến `JAVA_HOME` đã thiết lập đúng
- ☐ Docker chạy được lệnh `docker run hello-world` thành công
- ☐ Git đã cấu hình, đã tạo repository cho bài tập
- ☐ Hadoop (Pseudo-Distributed) chạy được, HDFS Web UI truy cập được
- ☐ Spark chạy được ví dụ WordCount đầu tiên
- ☐ Đã đọc trước tài liệu Han–Kamber–Pei, Chương 1 cho buổi học Tuần 2

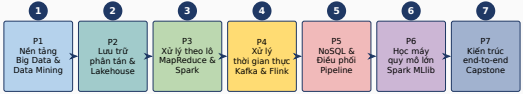
Tổng kết Tuần 1

- Ngữ cảnh & Ứng dụng
  - 8 lĩnh vực nơi Big Data thực sự tạo ra giá trị, và đặc trưng 5V nào nổi bật ở mỗi lĩnh vực
- Roadmap 7 phần
  - Logic kết nối xuyên suốt: Nền tảng → Lưu trữ → Xử lý → (Streaming / NoSQL) → Học máy → Kiến trúc tổng thể
- Thực hành
  - Môi trường làm việc (Python, Java, Docker, Git, Hadoop, Spark) đã sẵn sàng cho 8 tuần tiếp theo

Chuẩn bị cho Tuần 2 — Phần 1: Nền tảng Big Data & Data Mining

- Đọc trước: Han, Kamber, Pei — *Data Mining: Concepts and Techniques*, Chương 1
- Mang theo laptop đã hoàn thành checklist cài đặt ở phần thực hành hôm nay
- Chuẩn bị sẵn ít nhất một ý tưởng đề tài khả dĩ cho dự án capstone (sẽ chốt chính thức ở Tuần 2–3)
- Ôn lại 3 câu hỏi thảo luận ở cuối mỗi phần nếu muốn chuẩn bị kỹ hơn trước khi vào Tuần 2

Hỏi & Đáp



Hẹn gặp lại ở Tuần 2 — Phần 1: Nền tảng Big Data & Data Mining

Tài liệu tham khảo tổng hợp cả khóa học được liệt kê ở cuối mỗi phần trong slide chi tiết